

Contents

Chapter 21: The Agentic Software Project	2
1. The Traditional Development Model	2
2. The Agentic Development Model	3
3. The Week 1 Application Becomes the Target	4
4. Give the Agent Ownership of a Development Objective	5
5. The Engineer Becomes the Specification Layer	5
6. Start With Architecture, Not Code	6
7. Repository Understanding	7
8. The First Assignment: Redesign	8
9. Architecture Decision Records	9
10. The Second Assignment: Add Features	9
11. Feature Development Should Be Specification-Driven	10
12. The Third Assignment: Testing	11
13. Tests as Executable Specification	11
14. The Fourth Assignment: Benchmark	12
15. Benchmark Before Modification	12
16. The Agent as an Optimization Loop	13
17. The Fifth Assignment: Bug Fixing	13
18. Root Cause Analysis	14
19. The Sixth Assignment: Documentation	15
20. Documentation Drift as a Testable Property	15
21. Your Role Changes	16
22. The Engineer as Control System	17
23. The Agent Is an Actuator	18
24. The Agentic Development Loop	18
25. Human-in-the-Loop Does Not Mean Human-at-Every-Step	19
26. Delegation Requires Clear Boundaries	20
27. The Agent Should Produce Evidence	21
28. Evidence-Based Acceptance	21
29. What You Should Not Accept	22
30. A Full Project Assignment	23
31. The Agent’s Deliverables	24
32. Evaluate the Agent, Not Just the Application	24
33. Agent Engineering Metrics	25
34. Measure Human Leverage	26
35. The Engineer Moves Up the Abstraction Stack	26
36. This Is Not “No-Code”	27
37. The Verification Burden Increases	27
38. Specification Becomes a Force Multiplier	28
39. The Agentic Engineer as a Systems Architect	28
40. The Meta-Level: Engineering the Engineering Process	29
41. The Week 1 Project Revisited	30
42. The Chapter 21 Challenge	30
43. Debug the Development System	31

44. A Mature Chapter 21 Workflow	32
45. Key Takeaways	33

Chapter 21: The Agentic Software Project

Chapter 21 is the culmination of the first major phase of the bootcamp.

The previous chapters developed the individual capabilities required for agentic software engineering:

```

Chapter 15 - Coding agents
↓
Chapter 16 - Specification engineering
↓
Chapter 17 - Context engineering
↓
Chapter 18 - Agentic development loops
↓
Chapter 19 - Multi-agent systems
↓
Chapter 20 - Coding-agent safety
↓
Chapter 21 - Agentic software project

```

Until now, the exercises have isolated individual concepts.

Today they become one engineering system.

You take the application built during Week 1 and give a coding agent substantial responsibility for its evolution.

The objective is deliberately different from a traditional programming assignment.

You are not primarily practicing:

How to write more code.

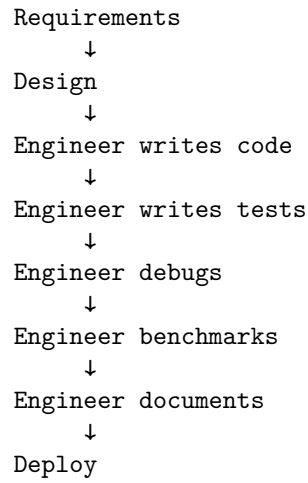
You are practicing:

How to engineer software when an AI system performs much of the implementation work.

That changes the engineer's role.

1. The Traditional Development Model

The conventional workflow looks approximately like:



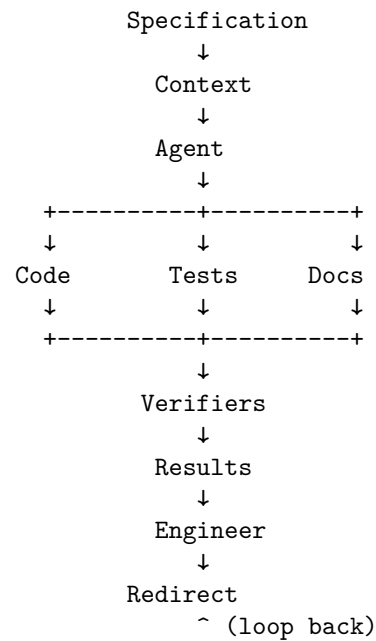
The engineer is the primary producer of implementation.

The tools accelerate the engineer.

The engineer remains in the center of the implementation loop.

2. The Agentic Development Model

With a capable coding agent, the workflow changes:



The engineer increasingly becomes the **controller of the development process** rather than its primary typist.

Your job becomes:

Specify
Review
Test
Evaluate
Redirect
Architect

This is the central exercise of Chapter 21.

3. The Week 1 Application Becomes the Target

Return to the application built during Week 1.

The original project was a **Personal Research Assistant** with capabilities such as:

- document ingestion
- retrieval
- question answering
- citations
- tool use
- conversational state
- uncertainty detection
- structured outputs
- evaluation
- metrics
- deployment

The important point is that this application already exists.

You are no longer starting from:

blank repository

You are starting from:

existing system
+
existing architecture
+
existing tests
+
existing bugs

+
existing technical debt

That makes the exercise much closer to real software engineering.

4. Give the Agent Ownership of a Development Objective

Do not simply tell the agent:

“Improve this application.”

That is too vague.

Instead, specify an engineering objective.

For example:

Redesign the retrieval subsystem to improve answer groundedness while maintaining the existing API contract. Preserve backward compatibility. Add regression tests for existing behavior. Achieve at least a 10% improvement on the retrieval evaluation suite without increasing p95 latency by more than 20%.

Now the agent has:

Goal
Constraints
Interfaces
Metrics
Acceptance criteria

This connects directly to Chapter 16.

5. The Engineer Becomes the Specification Layer

A useful way to think about the new workflow is:

Engineer
↓
Specification
↓
Agent
↓
Implementation

The engineer’s leverage increasingly comes from the quality of the specification.

A weak specification:

Improve retrieval.

A stronger specification:

Goal:

Improve retrieval quality.

Constraints:

- preserve public API
- preserve citation format
- no new external database
- p95 latency < 800 ms

Acceptance criteria:

- Recall@10 $\geq$ 0.90
- groundedness $\geq$ 0.92
- existing tests remain green
- no regression in citation accuracy

Deliverables:

- implementation
- tests
- benchmark results
- architecture decision record
- documentation

The second specification gives the agent a much more constrained search space.

6. Start With Architecture, Not Code

One of the most important rules for this exercise is:

Do not immediately ask the agent to modify the repository.

First ask it to understand the system.

For example:

Analyze this repository.

Produce:

1. architecture map
2. module dependency graph
3. major data flows
4. public interfaces
5. test architecture
6. evaluation architecture

7. performance bottlenecks
8. technical debt
9. security risks
10. recommended improvement areas

Do not modify any files.

This is context engineering in practice.

The agent must construct a model of the system before changing it.

7. Repository Understanding

A useful coding agent should be able to answer questions such as:

Where is retrieval implemented?

Where are documents ingested?

Where is conversation state stored?

Where are citations generated?

Where are model calls made?

Where are evaluation datasets defined?

Which modules are public interfaces?

Which tests protect those interfaces?

Where are the performance bottlenecks?

The agent's first deliverable should therefore often be a **repository map**.

For example:

```
src/
+-- api/
|   +-- routes.py
+-- ingestion/
|   +-- parser.py
|   +-- chunker.py
+-- retrieval/
|   +-- embed.py
|   +-- search.py
|   +-- rerank.py
```

```
+-- generation/  
|   +-- answer.py  
+-- memory/  
|   +-- conversation.py  
+-- evaluation/  
|   +-- dataset.py  
|   +-- metrics.py  
+-- tools/  
    +-- search.py
```

This map becomes part of the agent's working context.

8. The First Assignment: Redesign

The first major task is:

Have the agent redesign the application.

But redesign should not mean:

Rewrite everything.

Instead:

```
Current architecture  
  ↓  
Identify weaknesses  
  ↓  
Propose alternatives  
  ↓  
Evaluate tradeoffs  
  ↓  
Select architecture  
  ↓  
Implement incrementally
```

Require the agent to explain:

- why the current design is insufficient
 - what the proposed architecture changes
 - what interfaces change
 - what risks are introduced
 - what migration strategy is required
 - how the new design will be verified
-

9. Architecture Decision Records

Require the agent to produce an ADR.

For example:

ADR-007: Introduce hybrid retrieval

Status:

Accepted

Context:

Semantic retrieval misses exact identifiers and technical terms.

Decision:

Combine BM25 and embedding retrieval followed by reranking.

Alternatives:

1. embeddings only
2. BM25 only
3. hybrid retrieval

Consequences:

- + improved recall
- + better exact-match behavior
- additional latency
- additional infrastructure

This forces the agent to reason about architecture rather than merely generate code.

10. The Second Assignment: Add Features

Next, have the agent add meaningful capabilities.

For example:

Feature:

Support document collections.

Requirements:

- users can create collections
- documents belong to collections
- retrieval can be scoped to a collection
- citations identify collection and document
- existing API behavior remains compatible

The agent should produce:

implementation

+

tests

+

documentation

+

evaluation

Do not accept:

implementation only

The artifact is the complete engineering change.

11. Feature Development Should Be Specification-Driven

A useful interaction pattern is:

Engineer:

specification

Agent:

implementation plan

Engineer:

review / approve

Agent:

implementation

Agent:

tests

Agent:

verification

Engineer:

evaluate

Agent:

fix

Notice that the agent does not immediately jump from:

requirement

to:

code

The planning stage provides a control point.

12. The Third Assignment: Testing

Now explicitly ask the agent to improve the test suite.

The agent should identify:

- missing unit tests
- integration tests
- regression tests
- edge cases
- failure modes
- API contract tests
- security tests
- evaluation cases

A strong assignment might be:

Analyze the current implementation and identify important behaviors that are not covered by tests. Add tests for those behaviors without weakening existing assertions.

This is more interesting than:

Write more tests.

The objective is **coverage of meaningful behavior**, not test-count maximization.

13. Tests as Executable Specification

This connects Chapter 21 directly to specification engineering.

A requirement might state:

Retrieval must never return documents from another tenant.

The corresponding test becomes:

```
Tenant A query
  ↓
retrieve
  ↓
assert:
all returned documents in Tenant A
```

Now the requirement is executable.

The architecture therefore becomes:

Specification

↓

Tests

↓

Implementation

↓

Verification

This creates a powerful feedback mechanism for the agent.

14. The Fourth Assignment: Benchmark

Next, tell the agent:

Measure the system before and after your changes.

This is critical.

Without measurement:

"the new implementation is better"

is merely a claim.

With measurement:

Baseline:

p95 latency = 620 ms

Recall@10 = 0.81

After:

p95 latency = 690 ms

Recall@10 = 0.91

Now the engineering tradeoff is visible.

15. Benchmark Before Modification

Always establish a baseline.

For example:

Baseline

↓

+-----+-----+

↓ ↓ ↓
accuracy latency cost

Record:

- throughput
- latency
- p50
- p95
- p99
- memory
- CPU/GPU utilization
- token usage
- inference cost
- retrieval metrics
- answer quality

Then make the change.

Then measure again.

16. The Agent as an Optimization Loop

The workflow becomes:

Baseline
↓
Hypothesis
↓
Implementation
↓
Benchmark
↓
Compare
↓
Accept / reject

The agent is no longer merely generating code.

It is participating in an empirical optimization process.

This is a major conceptual shift.

17. The Fifth Assignment: Bug Fixing

Now deliberately introduce bugs.

For example:

- broken citation handling
- incorrect chunk boundaries
- stale conversation state
- race condition
- incorrect authorization
- malformed structured output
- retrieval regression

Give the agent the failing tests or observed symptoms.

Do not tell it exactly where the problem is.

Ask it to:

1. reproduce the failure
2. diagnose the root cause
3. propose a fix
4. implement the fix
5. add a regression test
6. rerun verification

This tests actual engineering ability.

18. Root Cause Analysis

A good agent should not merely patch symptoms.

Require:

Failure

↓

Reproduction

↓

Hypothesis

↓

Evidence

↓

Root cause

↓

Fix

↓

Regression test

For example:

Symptom:

Citation missing in 7% of answers.

Root cause:
Citation metadata is discarded during reranking.

Fix:
Preserve metadata through retrieval pipeline.

Regression test:
assert citation metadata survives reranking.

This is substantially more valuable than:

“I changed the citation code and the test passes.”

19. The Sixth Assignment: Documentation

Documentation should also become part of the agent’s responsibilities.

Ask the agent to update:

- README
- architecture documentation
- API documentation
- configuration documentation
- deployment instructions
- troubleshooting guides
- ADRs
- examples

The agent has an important advantage here:

It has already inspected the implementation.

The goal is to ensure that:

Implementation

<->

Documentation

remain consistent.

20. Documentation Drift as a Testable Property

Documentation is often treated as secondary.

In agentic engineering, it can become part of the verification loop.

For example:

API specification

↓

implementation

↓

API tests

↓

documentation

A CI check might verify that:

- documented endpoints exist
- examples execute
- schemas match
- configuration options are valid

The agent can then repair documentation when implementation changes.

21. Your Role Changes

The most important exercise is not actually what the agent does.

It is what **you stop doing**.

You should deliberately avoid becoming the primary coder.

Instead, your role becomes:

Specify Define:

what
why
constraints
interfaces
acceptance criteria

Review Evaluate:

architecture
implementation plan
tradeoffs
risks

Test Determine:

what must be true
what can fail
what evidence is required

Evaluate Measure:

quality
performance
cost
reliability
security

Redirect When the agent goes down the wrong path:

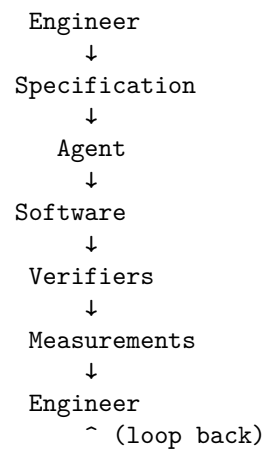
stop
diagnose
clarify
redirect

Architect Decide:

system boundaries
interfaces
data flows
technology choices
invariants

22. The Engineer as Control System

A useful abstraction is to think of yourself as the controller.



This resembles a feedback-control system.

The agent is the actuator.

The software is the system being changed.

The tests and evaluations are sensors.

The engineer is the controller.

The specification defines the desired state.

23. The Agent Is an Actuator

This is an important conceptual shift.

Traditional software engineering:

Engineer → Code

Agentic engineering:

Engineer
↓
Desired state
↓
Agent
↓
Actions
↓
System

The agent executes a policy toward the desired state.

This means the engineer's leverage increasingly comes from controlling:

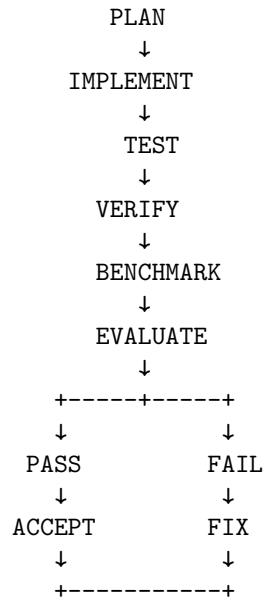
- objective
- state
- context
- tools
- constraints
- feedback
- stopping conditions

rather than manually specifying every implementation detail.

24. The Agentic Development Loop

By Chapter 21, your complete loop should look like:

SPECIFICATION
↓
CONTEXT
↓



The human engineer supervises the loop.

The coding agent performs much of the execution.

25. Human-in-the-Loop Does Not Mean Human-at-Every-Step

A common mistake is to interpret agentic development as:

Agent:
"Should I edit file A?"

Human:
"Yes."

Agent:
"Should I run pytest?"

Human:
"Yes."

Agent:
"Should I fix the failure?"

Human:
"Yes."

This provides little leverage.

The objective is instead:

Human:

Here is the specification,
constraints,
acceptance criteria,
and approval boundary.

Agent:

Execute.

Verifier:

Measure.

Agent:

Fix failures.

Human:

Review the resulting evidence.

The human operates at the level of **decisions and boundaries**, not keystrokes.

26. Delegation Requires Clear Boundaries

You should explicitly define what the agent can decide.

For example:

Agent may decide

implementation details
file organization
refactoring strategy
test structure
local optimization

Engineer decides

public API changes
architecture changes
security model
database schema strategy
production deployment
major dependencies
cost-risk tradeoffs

This creates a delegation boundary.

27. The Agent Should Produce Evidence

One of the most important requirements for Chapter 21 is:

Do not accept “done” as a status. Require evidence.

Instead of:

“The feature is implemented.”

require:

Implementation:
complete

Tests:
142 passed

New tests:
17

Evaluation:
Recall@10 +8.4%

Latency:
p95 +6%

Security:
no new findings

Documentation:
updated

Known limitations:
...

The agent’s output becomes an **engineering report**, not merely a claim.

28. Evidence-Based Acceptance

A useful acceptance structure is:

Requirement
↓

Evidence

↓

Decision

For example:

Requirement	Evidence	Result
API remains backward compatible	Contract tests	PASS
Recall@10 ≥ 0.90	Evaluation suite	PASS
p95 < 800 ms	Benchmark	PASS
No cross-tenant retrieval	Security tests	PASS
Documentation updated	Documentation checks	PASS

This makes the agent’s work auditable.

29. What You Should Not Accept

Do not accept:

“I think this should work.”

Require:

“The test suite demonstrates X.”

Do not accept:

“This should improve performance.”

Require:

“Benchmark results show a 17% reduction in p95 latency.”

Do not accept:

“The architecture is cleaner.”

Require:

“The new architecture removes the dependency cycle and reduces module coupling from X to Y.”

The distinction is:

assertion $\rightarrow$ evidence

30. A Full Project Assignment

Give the agent the following objective:

Take the Week 1 Personal Research Assistant and evolve it into a production-quality agentic application.

The agent must:

Architecture

- analyze the existing architecture
- identify weaknesses
- propose improvements
- produce ADRs
- implement approved architectural changes

Features

- implement at least two significant new capabilities
- preserve existing public interfaces unless explicitly approved
- update configuration and deployment

Testing

- add unit tests
- add integration tests
- add regression tests
- improve evaluation coverage
- add failure-mode tests

Performance

- establish baseline
- benchmark changes
- identify bottlenecks
- optimize at least one subsystem

Reliability

- add retries where appropriate
- handle failures
- improve observability
- define stopping conditions

Security

- inspect tool permissions
- identify prompt-injection risks

- verify secret isolation
- test authorization boundaries

Documentation

- update README
 - update architecture documentation
 - update API documentation
 - add ADRs
 - document deployment
-

31. The Agent's Deliverables

Require a complete artifact set:

```
project/
+-- source code
+-- tests/
+-- evaluations/
+-- benchmarks/
+-- docs/
+-- ADRs/
+-- architecture diagram
+-- security analysis
+-- engineering report
```

The final report should contain:

1. What changed?
2. Why?
3. What alternatives were considered?
4. What tests were added?
5. What benchmarks were run?
6. What improved?
7. What regressed?
8. What risks remain?
9. What decisions require human review?
10. What should be done next?

This is much closer to professional engineering practice than a code-generation demo.

32. Evaluate the Agent, Not Just the Application

There are now two things to evaluate.

Application quality

Does the software work?

Agent quality

Can the agent reliably modify the software?

These are different.

An application can succeed even if the agent required enormous human intervention.

Conversely, an agent can appear autonomous while producing poor software.

Measure both.

33. Agent Engineering Metrics

Useful metrics include:

Task success rate

$$P(\text{task completed successfully})$$

Intervention rate

$$\frac{\text{human interventions}}{\text{agent tasks}}$$

First-pass success

$$P(\text{success without human correction})$$

Iterations per task

$$N_{\text{iterations}}$$

Regression rate

$$P(\text{existing behavior broken})$$

Cost per successful task

$$\frac{\text{inference} + \text{infrastructure cost}}{\text{successful tasks}}$$

Time to verified completion

$$T_{\text{verified}}$$

These metrics allow you to evaluate whether the agent is actually becoming useful as an engineering collaborator.

34. Measure Human Leverage

One of the most interesting metrics is:

$$L = \frac{\text{software value produced}}{\text{human implementation effort}}$$

Traditional development might look like:

Human:
100 units implementation
Agent:
20 units assistance

An agentic workflow might look like:

Human:
20 units specification/review
Agent:
100 units implementation

The objective is not to eliminate the engineer.

It is to increase the engineer's leverage.

35. The Engineer Moves Up the Abstraction Stack

The progression looks approximately like:

Architecture
^
Specification
^
Design
^
Implementation
^
Code

As agents become better at implementation, the human can spend more time near the top.

This does not make implementation knowledge irrelevant.

Quite the opposite.

You need enough technical depth to determine whether the agent’s implementation is sound.

But you increasingly exercise that knowledge through:

- architecture
- constraints
- review
- evaluation
- debugging
- system-level decisions

rather than line-by-line typing.

36. This Is Not “No-Code”

There is an important distinction.

Agentic engineering is not:

“I don’t need to understand programming because the AI writes the code.”

It is closer to:

“I need to understand software deeply enough to specify, evaluate, constrain, and correct software that an AI produces.”

The implementation burden decreases.

The responsibility for correctness does not.

In some respects, it increases.

37. The Verification Burden Increases

Suppose an engineer manually writes 500 lines of code.

They have direct knowledge of what they wrote.

Now an agent produces 5,000 lines.

The engineer may have much less direct familiarity with every implementation detail.

Therefore:

More generated code

↓

Greater verification requirement

This is why:

Generation capability $\Rightarrow$ Verification capability

must grow together.

38. Specification Becomes a Force Multiplier

A poorly specified task:

"Improve the application."

can produce enormous amounts of plausible but unnecessary work.

A precise specification:

Goal

Constraints

Interfaces

Invariants

Acceptance criteria

Tests

Benchmarks

constrains the agent's search space.

The engineer's specification therefore becomes a form of **programming at a higher level of abstraction**.

39. The Agentic Engineer as a Systems Architect

The engineer's responsibilities increasingly become:

Problem definition

↓

Specification

↓

Architecture

↓
Agent delegation
↓
Verification
↓
Evaluation
↓
Decision

This resembles architecture more than traditional implementation.

The engineer is designing not just the software, but the **process by which the software is produced**.

That is a major conceptual shift.

40. The Meta-Level: Engineering the Engineering Process

Traditional software engineering asks:

How should we build this system?

Agentic software engineering increasingly asks two questions:

How should we build this system?

and:

How should we design the agentic process that builds this system?

For example:

Application architecture
+
Agent architecture
+
Context architecture
+
Verification architecture
+
Security architecture

The engineering object is now larger than the application itself.

41. The Week 1 Project Revisited

At the beginning of the bootcamp, the Personal Research Assistant was primarily an application.

By Chapter 21, it becomes a testbed for an engineering methodology.

You now have:

```
Application
+
Coding Agent
+
Specification
+
Context
+
Tools
+
Verification
+
Evaluation
+
Security
+
Human oversight
```

This is the first complete **agentic software-development system** in the curriculum.

42. The Chapter 21 Challenge

For the final challenge, impose one additional constraint:

You are not allowed to directly implement a feature unless the agent has failed to implement it correctly after a reasonable number of iterations.

This forces you to practice the new workflow.

When something fails, your first response should not be:

"I'll just fix it myself."

Instead:

Why did the agent fail?

↓

Was the specification insufficient?

↓

Was the context insufficient?
↓
Was the architecture unclear?
↓
Was the verifier inadequate?
↓
Was the task decomposition wrong?
↓
Redirect the agent.
This turns agent failure into an engineering diagnostic.

43. Debug the Development System

This is one of the most important lessons.
Suppose the agent repeatedly introduces incorrect changes.
There are several possible causes:

Bad specification
↓
Bad implementation

or:

Bad context
↓
Bad implementation

or:

Bad architecture
↓
Bad implementation

or:

Weak tests
↓
Bad implementation survives

or:

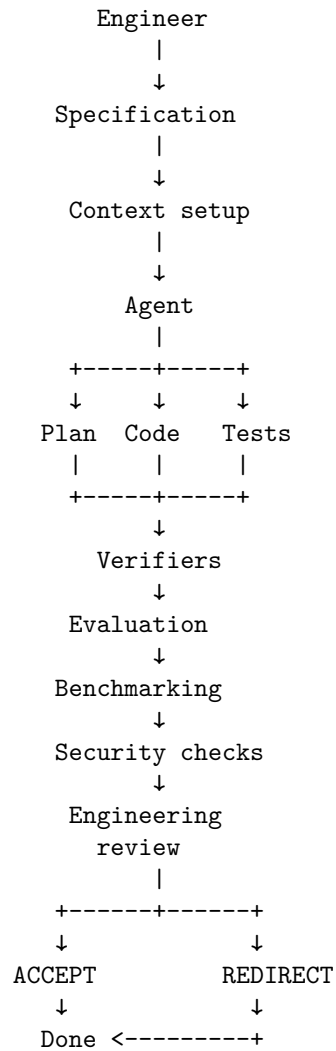
Poor tool configuration
↓
Agent cannot inspect necessary state

Therefore:

When an agent fails, debug the entire development system—not just the generated code.

44. A Mature Chapter 21 Workflow

The final workflow should look something like:



The human is not in every loop iteration.

The human controls the system and intervenes at meaningful decision boundaries.

45. Key Takeaways

1. **Chapter 21 is the transition from learning agent capabilities to practicing agentic engineering.**
2. **The Week 1 application becomes the subject of autonomous evolution.** The agent should redesign it, add features, write tests, benchmark it, fix defects, and maintain documentation.
3. **Your role changes from primary implementer to engineering controller:**
 - Specify
 - Review
 - Test
 - Evaluate
 - Redirect
 - Architect
4. **Do not begin by asking the agent to code.** Have it first understand the repository, architecture, interfaces, tests, constraints, and technical debt.
5. **Specification becomes the primary control surface.**
 - Goal
 - + constraints
 - + interfaces
 - + invariants
 - + acceptance criteria
 - + tests
 - + benchmarks
6. **Require plans before large implementation changes.** Planning creates an important human review boundary.
7. **Require evidence, not assertions.**
 - "It works"
 - is weak.
 - "142 tests pass, Recall@10 increased from .81 to .91, p95 latency increased 6%, and no security regressions were detected."
 - is engineering evidence.
8. **Testing, benchmarking, security, and documentation are part of the implementation—not post-processing.**

9. **Agent failures should be diagnosed at the system level.** Investigate specification, context, architecture, tools, decomposition, and verification—not merely the generated code.
10. **Verification becomes more important as generation becomes cheaper.**

More generation $\Rightarrow$ More need for verification

11. **Agentic development is not “no-code.”** It requires deep engineering knowledge because someone must establish architecture, constraints, correctness criteria, and acceptance decisions.
12. **The engineer moves upward in the abstraction hierarchy.**

```

Code
~
Implementation
~
Design
~
Specification
~
Architecture
~
Engineering process

```

13. **The engineer increasingly designs the process that produces the software, not just the software itself.**
14. **Measure agent performance as well as application performance.** Important metrics include task success, intervention rate, first-pass success, iterations, regressions, cost, and time-to-verified-completion.
15. **The ultimate objective is human leverage.**

$$\text{Engineering leverage} = \frac{\text{verified software value}}{\text{human implementation effort}}$$

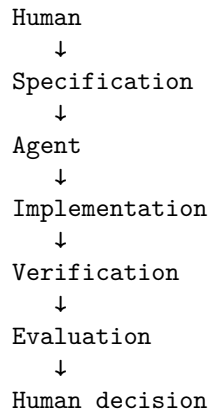
16. **The deepest lesson of Chapter 21 is that AI-assisted development is not primarily about typing code faster.**

It is about changing the unit of engineering work:

Traditional:

Human $\rightarrow$ Code

Agentic:



17. **The engineer becomes the architect of a software-producing system.**

This is why Chapter 21 is such an important transition point in the bootcamp. You have spent the first three weeks learning how to make an agent capable, how to specify its work, how to provide context, how to create feedback loops, how to coordinate agents, and how to constrain their authority.

Now you put all of those ideas together.

The objective is not to prove that an AI can write code.

That question is already becoming less interesting.

The important question is:

Can an engineer construct a specification, context, tool environment, feedback system, verification pipeline, and governance boundary in which an AI can reliably produce and evolve production-quality software?

That is the beginning of **agentic software engineering**.